HelixCode Anti-Bluff Testing Strategy

Purpose

This document defines the comprehensive testing strategy that ensures **every passing test and challenge is a GUARANTEE that the tested feature actually works for end users**. This strategy is mandatory per HelixCode Constitution CONST-035 (End-User Usability Mandate).

Core Principle: A test that passes MUST certify:

- **Quality** - Feature behaves correctly under real user inputs, edge cases, and concurrency
- **Completion** - Feature is wired end-to-end from API to infrastructure, no stub gaps
- **Full Usability** - A user following documentation succeeds without internal knowledge

---

1. Test Pyramid (Anti-Bluff Edition)

```
        /\
       /  \     E2E Challenges (Real projects, real providers)
      / __ \    Integration Tests (Real DB, real HTTP, real containers)
     / /  \ \   Contract Tests (Provider API compatibility)
    / / __ \ \  Component Tests (Real subsystems wired together)
   /_/ /  \ \_\ Unit Tests (Mocks ALLOWED here only)
  /___/    \___\ Static Analysis (Lint, security, structure)
```

Anti-Bluff Rule: Unit tests are the ONLY level where mocks/stubs are permitted. Every other level MUST use real infrastructure.

---

2. Test Categories & Requirements

2.1 Unit Tests (`*_test.go` with `-short` flag)

Scope: Individual functions, methods, pure logic **Mock Policy**: MOCKS ALLOWED **Execution**: `go test -short ./...` **Resource Limits**: `GOMAXPROCS=2`, `nice -n 19`

Requirements:

- [ ] Table-driven tests with subtests (`t.Run()`)
- [ ] Use `testify/require` for fatal assertions
- [ ] Use `testify/assert` for non-fatal checks
- [ ] Mock external dependencies ONLY
- [ ] Coverage target: 80%+ per package

Example:

```
func TestAuthService_Register(t *testing.T) {
    tests := []struct {
        name     string
        username string
        email    string
        password string
        wantErr  bool
    }{
        {"valid", "user", "user@example.com", "password123", false},
        {"short_password", "user", "user@example.com", "123", true},
        {"invalid_email", "user", "not-an-email", "password123", true},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            mockRepo := new(MockAuthRepository)
            svc := auth.NewAuthService(auth.DefaultConfig(), mockRepo)

            _, err := svc.Register(context.Background(), tt.username, tt.email,
tt.password, "")
            if tt.wantErr {
                require.Error(t, err)
            } else {
                require.NoError(t, err)
            }
        })
    }
}
```

2.2 Contract Tests

Scope: LLM provider API compatibility, external service contracts **Mock Policy**: NO MOCKS - test against real or recorded APIs **Execution**: `go test ./tests/contract/...`

Requirements:

- [ ] Each provider has contract tests against real API
- [ ] Tests validate request/response schemas
- [ ] Tests check error codes and handling
- [ ] Recorded responses (vcr-style) for CI where live API unavailable
- [ ] MUST skip (not fail) if API unavailable

Example:

```
func TestOllamaProvider_Contract(t *testing.T) {
    if testing.Short() {
        t.Skip("Contract tests skipped in short mode")
    }

    provider := ollama.New(os.Getenv("OLLAMA_HOST"))
```

```
    if err := provider.HealthCheck(context.Background()); err != nil {
        t.Skipf("Ollama not available: %v", err)
    }

    // ANTI-BLUFF: Real call to real Ollama
    resp, err := provider.Generate(context.Background(), &llm.GenerateRequest{
        Prompt: "Say 'contract test passed'",
        Model:  "llama3.2",
    })

    require.NoError(t, err)
    require.NotEmpty(t, resp.Text)
    require.NotContains(t, resp.Text, "simulated")  // ANTI-BLUFF CHECK
}
```

2.3 Component Tests

Scope: Real subsystems wired together (DB + Auth, Worker + Task, etc.) **Mock Policy**: NO MOCKS - use test databases, test containers **Execution**: `go test ./tests/component/...` **Infrastructure**: Testcontainers for PostgreSQL, Redis

Requirements:

- ☐ Use real PostgreSQL (testcontainers or dedicated test DB)
- ☐ Use real Redis (testcontainers or dedicated test instance)
- ☐ Clean state between tests (truncate tables, flush Redis)
- ☐ Test actual HTTP handlers with `httptest.Server`
- ☐ Verify real data persistence

Example:

```
func TestTaskLifecycle_Component(t *testing.T) {
    ctx := context.Background()

    // Start real PostgreSQL container
    postgresC := startTestPostgres(t)
    defer postgresC.Terminate(ctx)

    // Start real Redis container
    redisC := startTestRedis(t)
    defer redisC.Terminate(ctx)

    // Initialize real service with real DB
    db := connectToPostgres(postgresC)
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Create task in real DB
    task, err := taskMgr.Create(ctx, &task.Task{Title: "Test Task"})
    require.NoError(t, err)
    require.NotZero(t, task.ID)
```

```
    // ANTI-BLUFF: Verify task persisted
    persisted, err := taskMgr.Get(ctx, task.ID)
    require.NoError(t, err)
    require.Equal(t, "Test Task", persisted.Title)

    // ANTI-BLUFF: Start task execution
    err = taskMgr.Start(ctx, task.ID)
    require.NoError(t, err)

    // Verify state change in DB
    updated, err := taskMgr.Get(ctx, task.ID)
    require.NoError(t, err)
    require.Equal(t, task.StatusRunning, updated.Status)
}
```

2.4 Integration Tests

Scope: Full application with real external dependencies **Mock Policy**: NO MOCKS **Execution**: `make integration-test` or `./run_tests.sh --integration` **Infrastructure**: Full docker-compose stack

Requirements:

- ☐ Start full application stack (app + postgres + redis + nginx)
- ☐ Make real HTTP requests to running server
- ☐ Test authentication flow end-to-end (register -> login -> call API -> logout)
- ☐ Test LLM generation with real provider (or skip if unavailable)
- ☐ Test worker addition and task distribution
- ☐ Test notification dispatch (verify message arrives)
- ☐ Test file operations via API

Example:

```
func TestAPI_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration tests skipped in short mode")
    }

    baseURL := os.Getenv("HELIX_TEST_URL")
    if baseURL == "" {
        t.Skip("HELIX_TEST_URL not set")
    }

    client := &http.Client{Timeout: 30 * time.Second}

    // ANTI-BLUFF: Real registration
    resp, err := client.Post(baseURL+"/api/v1/auth/register",
        "application/json",

strings.NewReader(`{"username":"integuser","email":"integ@test.com","password":
"testpass123"}`))
    require.NoError(t, err)
```

```
    require.Equal(t, http.StatusCreated, resp.StatusCode)

    // ANTI-BLUFF: Real login
    resp, err = client.Post(baseURL+"/api/v1/auth/login",
        "application/json",
        strings.NewReader(`{"username":"integuser","password":"testpass123"}`))
    require.NoError(t, err)
    require.Equal(t, http.StatusOK, resp.StatusCode)

    var loginResp struct{ Token string }
    json.NewDecoder(resp.Body).Decode(&loginResp)
    require.NotEmpty(t, loginResp.Token)

    // ANTI-BLUFF: Real authenticated request
    req, _ := http.NewRequest("GET", baseURL+"/api/v1/workers", nil)
    req.Header.Set("Authorization", "Bearer "+loginResp.Token)
    resp, err = client.Do(req)
    require.NoError(t, err)
    require.Equal(t, http.StatusOK, resp.StatusCode)
}
```

2.5 E2E Challenges

Scope: Complete user workflows validating real-world usage **Mock Policy**: ABSOLUTELY NO MOCKS **Execution**: `tests/e2e/challenges/run_all_challenges.sh` **Infrastructure**: Production-like deployment

Challenge Requirements (per CONST-035):

1. Each challenge represents a REAL user workflow
2. Challenge MUST fail if feature is simulated or stubbed
3. Challenge MUST verify output quality (not just existence)
4. Challenge MUST use real infrastructure
5. Challenge wrapper MUST correctly propagate failures
6. Challenge MUST produce JSON report with pass/fail evidence

Challenge Categories

Category	Count	Description	Example
System Boot	5	Validate all services start and health checks pass	`full_system_boot_challenge.sh`
Constitution	3	Verify governance files exist and are valid	`constitution_validation_challenge.sh`
LLM Providers	15	Each provider generates real responses	`ollama_generation_challenge.sh`
Tools	20	Each tool performs real operations	`shell_execution_challenge.sh`

Category	Count	Description	Example
Workflows	10	Complete development workflows	`plan_build_test_challenge.sh`
Security	5	Auth, sandboxing, input validation	`auth_penetration_challenge.sh`
Performance	5	Load testing, resource limits	`concurrent_tasks_challenge.sh`
Anti-Bluff	10	Detect simulated/placeholder behavior	`no_simulation_challenge.sh`
Containers	5	Docker builds, compose orchestration	`docker_health_challenge.sh`

Challenge Framework

```
#!/bin/bash
# Challenge Framework Template
# Every challenge MUST follow this structure

set -euo pipefail

CHALLENGE_NAME="${1:-unknown}"
LOG_FILE="/tmp/challenge_${CHALLENGE_NAME}_$(date +%s).log"
RESULT_FILE="/tmp/challenge_results.json"

# ANTI-BLUFF: Robust failure tracking
PASSED=0
FAILED=0

log() {
    echo "[$(date -Iseconds)] $1" | tee -a "$LOG_FILE"
}

pass() {
    PASSED=$((PASSED + 1))
    log "|PASSED| $1"
}

fail() {
    FAILED=$((FAILED + 1))
    log "|FAILED| $1"
    # Continue running - don't exit immediately
}

cleanup() {
    # Generate JSON result
    cat >> "$RESULT_FILE" << EOF
    {
        "challenge": "$CHALLENGE_NAME",
```

```
        "timestamp": "$(date -Iseconds)",
        "passed": $PASSED,
        "failed": $FAILED,
        "total": $((PASSED + FAILED)),
        "success": $(if [ $FAILED -eq 0 ]; then echo "true"; else echo "false";
fi),
        "log": "$LOG_FILE"
    },
EOF
}
trap cleanup EXIT

# --- CHALLENGE BODY STARTS HERE ---

# ANTI-BLUFF: Always verify the real behavior, not just that code exists

# Example: LLM Generation Challenge
log "Testing LLM generation with Ollama..."
RESPONSE=$(curl -s -X POST http://localhost:8080/api/v1/llm/generate \
    -H "Content-Type: application/json" \
    -d '{"prompt":"What is the capital of France?","model":"llama3.2"}' \
    2>> "$LOG_FILE" || true)

# ANTI-BLUFF CHECK 1: Response must exist
if [ -z "$RESPONSE" ]; then
    fail "Empty response from LLM endpoint"
else
    pass "Received non-empty response"
fi

# ANTI-BLUFF CHECK 2: Response must not be simulated
if echo "$RESPONSE" | grep -qi "simulated"; then
    fail "Response contains 'simulated' - BLUFF DETECTED"
else
    pass "Response is not simulated"
fi

# ANTI-BLUFF CHECK 3: Response must actually answer the question
if echo "$RESPONSE" | grep -qi "paris"; then
    pass "Response correctly answers the question"
else
    fail "Response does not contain expected answer 'Paris'"
fi

# ANTI-BLUFF CHECK 4: Response must have reasonable length (not just echo)
RESPONSE_LEN=${#RESPONSE}
if [ "$RESPONSE_LEN" -lt 50 ]; then
    fail "Response too short ($RESPONSE_LEN chars) - likely just echoing
prompt"
else
    pass "Response has reasonable length ($RESPONSE_LEN chars)"
fi

# --- CHALLENGE BODY ENDS HERE ---
```

```
if [ $FAILED -gt 0 ]; then
    log "Challenge FAILED with $FAILED failures"
    exit 1
else
    log "Challenge PASSED with $PASSED checks"
    exit 0
fi
```

2.6 Security Tests

Scope: OWASP compliance, penetration testing, sandbox validation **Execution**: `make security-test` or `./run_tests.sh --security`

Requirements:

- [ ] SQL injection attempts against all endpoints
- [ ] Path traversal in file operations
- [ ] Command injection in shell tool
- [ ] JWT token tampering
- [ ] Rate limit enforcement
- [ ] CORS policy validation
- [ ] Sandbox escape attempts

2.7 Performance & Benchmarks

Scope: Response times, throughput, resource usage **Execution**: `make benchmark`

Requirements:

- [ ] LLM generation latency benchmarks
- [ ] Concurrent task handling benchmarks
- [ ] Database query performance benchmarks
- [ ] Memory usage profiling
- [ ] Worker scaling benchmarks

---

3. Test Execution Matrix

Test Type	Command	Environment	Duration	Mocks Allowed
Unit	`go test -short ./...`	Local	< 2 min	YES
Contract	`go test ./tests/contract/...`	Local + APIs	< 5 min	NO
Component	`go test ./tests/component/...`	Local + Testcontainers	< 10 min	NO

Test Type	Command	Environment	Duration	Mocks Allowed
Integration	`./run_tests.sh --integration`	Docker Compose	< 30 min	NO
E2E Challenges	`./run_all_challenges.sh`	Production-like	< 60 min	NO
Security	`./run_tests.sh --security`	Isolated	< 15 min	NO
Performance	`./run_tests.sh --benchmarks`	Dedicated	< 30 min	NO

4. Anti-Bluff Verification Checklist

For EVERY test added to HelixCode, verify:

- ☐ **Not a Wrapper Bluff**: Test wrapper correctly propagates all failures (use `grep "|FAILED|"` pattern, not just exit codes)
- ☐ **Not a Contract Bluff**: Test exercises the actual advertised capability, not a subset
- ☐ **Not a Structural Bluff**: Test verifies behavior, not just file/function existence
- ☐ **Not a Comment Bluff**: Test validates the actual code behavior, not the comment description
- ☐ **Not a Skip Bluff**: No bare `t.Skip()` without `SKIP-OK: #<ticket>` marker
- ☐ **Real Infrastructure**: Non-unit tests use real services (or properly skip if unavailable)
- ☐ **Real Assertions**: Assertions check actual output quality, not just non-nil/non-empty
- ☐ **Failure Testing**: Tests verify error handling with real failure injection
- ☐ **Concurrency Testing**: Tests verify thread-safety with goroutines
- ☐ **Resource Limits**: Tests respect GOMAXPROCS=2, nice -n 19 limits

5. Continuous Verification (Post-Implementation)

After every commit:

1. **Unit Tests**: `make test` - Must pass (2 min)
2. **Build Verification**: `make build` - Must compile all targets
3. **Lint & Format**: `make lint` - No warnings
4. **Integration Smoke**: `./run_tests.sh --smoke` - Core paths (5 min)
5. **Challenge Sample**: `./run_all_challenges.sh --sample` - 10% of challenges (5 min)

Before every release:

1. **Full Integration**: `./run_tests.sh --integration` - All integration tests
2. **Full Challenges**: `./run_all_challenges.sh` - ALL challenges
3. **Security Scan**: `./run_tests.sh --security`
4. **Performance Baseline**: `./run_tests.sh --benchmarks`
5. **Documentation Challenge**: Verify all README examples work

6. Test Data Management

6.1 Database Fixtures

- Use `testfixtures` or manual SQL scripts
- Load fixtures before component/integration tests
- Clean up (truncate) after each test
- NEVER use production data in tests

6.2 API Keys & Secrets

- Use `.env.test` for test credentials
- Mark tests to skip when credentials unavailable
- Use free-tier providers for CI (GitHub Copilot, Qwen, OpenRouter free models)
- Rotate test credentials monthly

6.3 Container Images

- Pin specific versions (not `latest`)
- Pre-pull images in CI to avoid network flakes
- Use minimal images where possible (alpine variants)

---

7. Failure Investigation Protocol

When a test or challenge fails:

1. **Reproduce locally**: Can you make it fail consistently?
2. **Check infrastructure**: Are all services running? (`docker-compose ps`)
3. **Check logs**: `docker-compose logs helixcode-server`
4. **Check resource limits**: Did the test exceed CPU/memory limits?
5. **Check for flakiness**: Run 10 times, does it fail consistently?
6. **Check anti-bluff indicators**: Does failure reveal a bluff?
7. **Document**: Add to `docs/issues/fixed/BUGFIXES.md` with reproduction steps

---

8. Success Metrics

Metric	Current	Target	Measurement
Unit Test Coverage	Unknown	80%+	`go test -cover`
Integration Test Count	Unknown	50+	Count in `tests/integration/`
Challenge Count	Unknown	100+	Count in `tests/e2e/challenges/`
Bluff Detection Rate	Unknown	100%	Challenges catch all simulated behavior
False Positive Rate	Unknown	0%	No test passes when feature is broken
CI Build Time	Unknown	< 10 min	`make ci-validate-all`

Metric	Current	Target	Measurement
Documentation Example Pass Rate	Unknown	100%	Every README example verified

This testing strategy is a living document. Every new feature MUST include tests that satisfy the anti-bluff criteria defined herein. No exceptions.